

Week5 Session 5 - Facilitation Guide

(Promises & Async/Await)

Index

- I. Recap of the previous session**
- II. Introduction to Promises (resolve, reject)**
- III. Chaining Promises**
- IV. Parallel Promises**
- V. async/await for Cleaner Asynchronous Code**

(2hr ILT)

I. Recap of the previous session

In the previous session, we delved into key foundational concepts of Node.js, focusing on its non-blocking architecture and its ability to handle multiple tasks concurrently. The session centered on understanding asynchronous programming and callbacks, which are fundamental to Node.js development. We explored the following key concepts:

Synchronous vs. Asynchronous Programming:

- We differentiated between synchronous programming, where tasks are executed sequentially, and asynchronous programming, where tasks can be executed concurrently without blocking the main thread.
- We discussed how Node.js leverages asynchronous programming to handle high levels of concurrency efficiently.

Callbacks in Node.js:

- We explored how callbacks function as a mechanism for handling asynchronous operations in Node.js.
- Callbacks were defined as functions passed as arguments to other functions, executed after an asynchronous task is completed.
- We practiced using callbacks in examples like reading files and handling events.

Callback Hell & How to Avoid It:

- We examined the phenomenon of "callback hell," where nested callbacks lead to complex, hard-to-read code.
- We discussed strategies to mitigate callback hell, such as modularizing code and using named functions.

By the end of the session, we gained a solid understanding of asynchronous programming using callbacks and the challenges associated with callback hell, setting the stage for more advanced asynchronous patterns.

In today's session, we will dive into Promises & Async/Await, powerful tools that simplify handling asynchronous code, making it more readable and manageable.

II. Introduction to Promises (resolve, reject)

A Promise is a modern JavaScript feature that helps manage asynchronous operations. It represents a value that may be available now, in the future, or never. In simple terms, a Promise is like a contract:

- It promises to return a result (value or error) once an asynchronous operation is completed.

In Node.js, Promises provide a cleaner and more structured way of handling asynchronous code, replacing older methods like callbacks that often lead to callback hell.

Why Do We Need Promises?

When working with asynchronous tasks, such as:

- Reading files
- Fetching data from an API
- Writing to a database

Using callbacks can make the code messy and hard to read. This is often referred to as callback hell, which occurs when you have nested callbacks, resulting in deeply indented and unreadable code.

Example of Callback Hell:

```
readFile("file1.txt", (err, data1) => {
  if (err) {
    console.error(err);
  } else {
    writeFile("file2.txt", data1, (err) => {
      if (err) {
        console.error(err);
      } else {
        console.log("File written successfully!");
      }
    });
  }
});
```

This becomes difficult to maintain as the complexity of the task increases. Promises solve this problem by flattening the code structure and improving readability.

Key Features of Promises

1. States of a Promise:

- Pending: The Promise is still in progress and hasn't been resolved or rejected yet.
- Fulfilled (Resolved): The asynchronous task is completed successfully.
- Rejected: The asynchronous task failed or encountered an error.

2. Methods for Handling Promises:

- .then(): Executes when the Promise is resolved.
- .catch(): Executes when the Promise is rejected.
- .finally(): Executes regardless of the Promise's outcome.

Anatomy of a Promise

A Promise is created using the Promise constructor, which takes a function called the executor as its argument.

The executor function has two parameters:

- resolve: A function to call when the operation is successful.
- reject: A function to call when the operation fails.

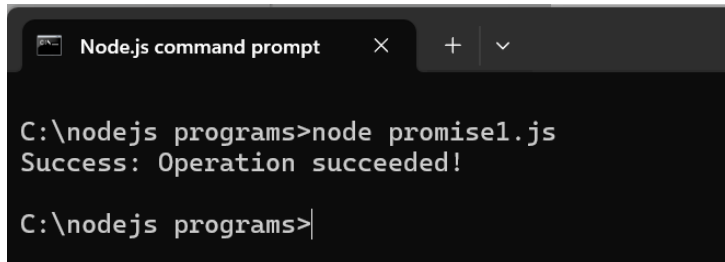
Example:

```
const myPromise = new Promise((resolve, reject) => {
  let success = true; // Simulating success or failure
  if (success) {
    resolve("Operation succeeded!");
  } else {
    reject("Operation failed!");
  }
});
```

```
});

// Consuming the Promise
myPromise
  .then((result) => {
    console.log("Success:", result);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise1.js
Success: Operation succeeded!
C:\nodejs programs>
```

How Promises Work (Step-by-Step)

1. A Promise starts in the pending state.
2. If the asynchronous task is successful, the resolve() function is called, and the Promise moves to the fulfilled state.
3. If the asynchronous task fails, the reject() function is called, and the Promise moves to the rejected state.
4. You use .then() to handle the resolved value and .catch() to handle errors.

Practical Example: Simulating an Asynchronous Task

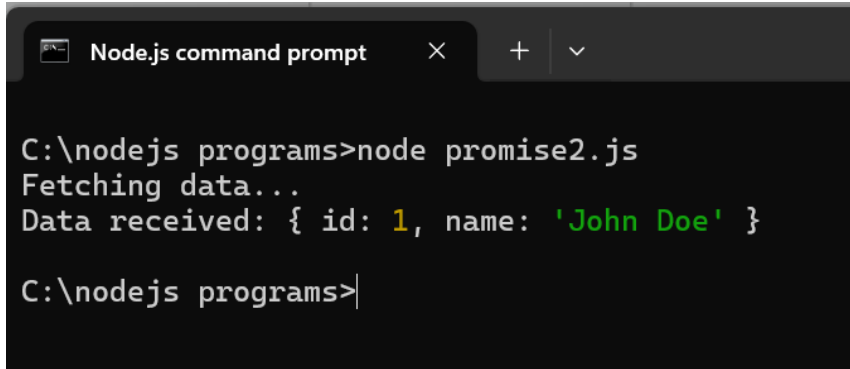
Let's simulate fetching data using Promises.

```
function fetchDataFromDataset() {
  return new Promise((resolve, reject) => {
    console.log("Fetching data...");
    setTimeout(() => {
      const data = { id: 1, name: "John Doe" };
      const error = false; // Simulating success or failure
      if (!error) {
        resolve(data);
      } else {
        reject("Error fetching data from the dataset!");
      }
    }, 2000); // Simulates a 2-second delay
  });
}
```

```
// Consuming the Promise
fetchDataFromDataset()
  .then((data) => {
    console.log("Data received:", data);
  })
  .catch((error) => {
```

```
console.error(error);
});
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise2.js
Fetching data...
Data received: { id: 1, name: 'John Doe' }
C:\nodejs programs>
```

Understanding resolve() and reject()

- resolve(value): Used to mark the Promise as fulfilled and pass the result to .then().
- reject(reason): Used to mark the Promise as rejected and pass the error to .catch().

Example:

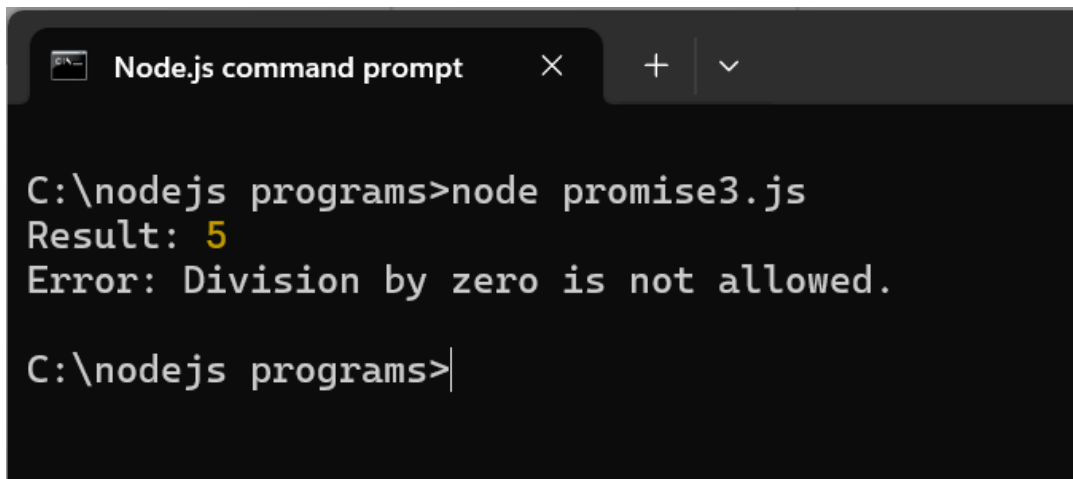
```
function divideNumbers(a, b) {
  return new Promise((resolve, reject) => {
    if (b !== 0) {
      resolve(a / b); // Successful division
    } else {
      reject("Division by zero is not allowed."); // Error
    }
  });
}
```

// Consuming the Promise

```
divideNumbers(10, 2)
  .then((result) => {
    console.log("Result:", result);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

```
divideNumbers(10, 0)
  .then((result) => {
    console.log("Result:", result);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command `C:\nodejs programs>node promise3.js` being executed. The output is `Result: 5` followed by an error message: `Error: Division by zero is not allowed.` The prompt is now at `C:\nodejs programs>` with a cursor.

```
C:\nodejs programs>node promise3.js
Result: 5
Error: Division by zero is not allowed.

C:\nodejs programs>
```

Real-Life Use Case: File System Example

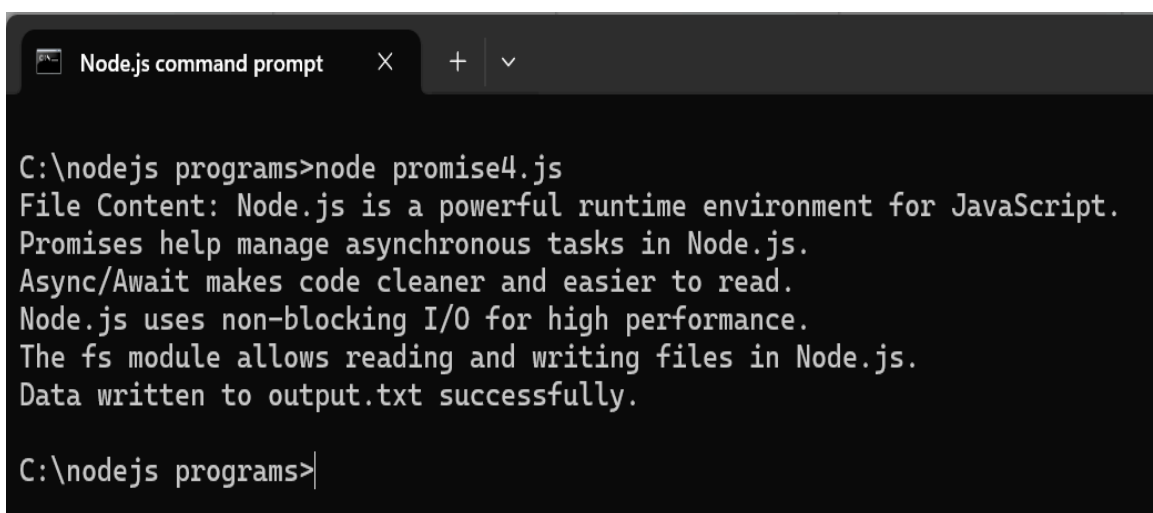
Let's use Promises to read and write files with Node.js.

Example:

```
const fs = require("fs").promises;

fs.readFile("input.txt", "utf8")
  .then((data) => {
    console.log("File Content:", data);
    return fs.writeFile("output.txt", data.toUpperCase());
  })
  .then(() => {
    console.log("Data written to output.txt successfully.");
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command `C:\nodejs programs>node promise4.js` being executed. The output is: `File Content: Node.js is a powerful runtime environment for JavaScript. Promises help manage asynchronous tasks in Node.js. Async/Await makes code cleaner and easier to read. Node.js uses non-blocking I/O for high performance. The fs module allows reading and writing files in Node.js. Data written to output.txt successfully.` The prompt is now at `C:\nodejs programs>` with a cursor.

```
C:\nodejs programs>node promise4.js
File Content: Node.js is a powerful runtime environment for JavaScript.
Promises help manage asynchronous tasks in Node.js.
Async/Await makes code cleaner and easier to read.
Node.js uses non-blocking I/O for high performance.
The fs module allows reading and writing files in Node.js.
Data written to output.txt successfully.

C:\nodejs programs>
```

Here, we used Promises to:

1. Read data from a file (input.txt).
2. Write the modified data to another file (output.txt).
3. Handle any potential errors.

Using `Promise.resolve()` and `Promise.reject()`

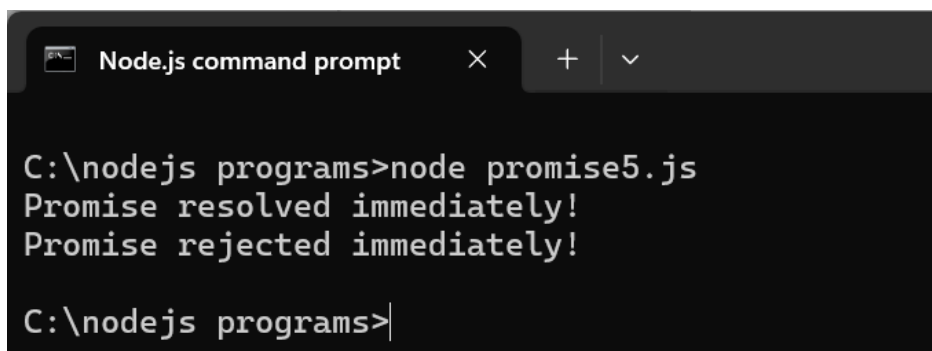
These are utility methods to create resolved or rejected Promises immediately.

Example:

```
// Immediately resolved Promise
Promise.resolve("Promise resolved immediately!")
  .then((message) => console.log(message));

// Immediately rejected Promise
Promise.reject("Promise rejected immediately!")
  .catch((error) => console.error(error));
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the following text:

```
C:\nodejs programs>node promise5.js
Promise resolved immediately!
Promise rejected immediately!

C:\nodejs programs>|
```

Advantages of Promises

1. Improved Code Readability: No more deeply nested callbacks.
2. Error Handling: Centralized error handling with `.catch()`.
3. Chaining: Allows sequential execution of asynchronous tasks with `.then()`.
4. Better Debugging: Easier to track and debug compared to callbacks.

Promises are an essential tool for modern JavaScript and Node.js development. They solve the problems associated with callback hell and make asynchronous code more readable and maintainable. By using `resolve`, `reject`, `.then()`, and `.catch()`, you can handle complex asynchronous tasks with ease.

III. Chaining Promises

Promise chaining is a technique that allows us to execute multiple asynchronous operations in a sequence, where the output of one operation can be passed as the input to the next. It is achieved by returning a Promise from a `.then()` block, enabling the next `.then()` in the chain to handle its resolved value.

This approach helps avoid the deeply nested structure of callbacks (callback hell) and creates more readable and maintainable asynchronous code.

How Promise Chaining Works

When a `.then()` returns a Promise, it passes the result of that Promise to the next `.then()` in the chain. If any error occurs in the chain, it propagates to the nearest `.catch()`.

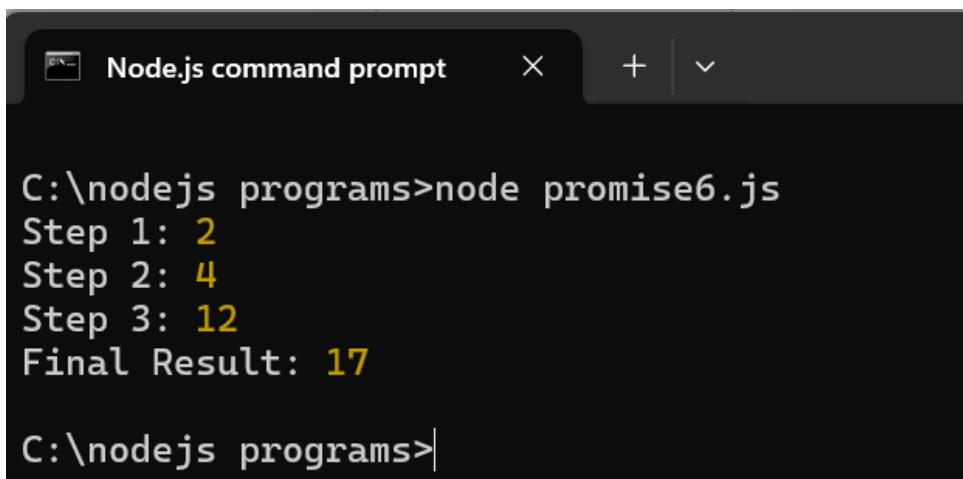
Key Concepts in Promise Chaining:

1. **Return values** in `.then()` are automatically wrapped in Promises.
2. Each `.then()` in the chain handles the resolved value of the previous Promise.
3. If an error occurs, the chain stops at the `.catch()` block.

Basic Example of Promise Chaining

```
new Promise((resolve, reject) => {
    resolve(2); // Start with 2
})
.then((result) => {
    console.log("Step 1:", result); // Step 1: 2
    return result * 2; // Pass 4 to the next `.then()`
})
.then((result) => {
    console.log("Step 2:", result); // Step 2: 4
    return result * 3; // Pass 12 to the next `.then()`
})
.then((result) => {
    console.log("Step 3:", result); // Step 3: 12
    return result + 5; // Pass 17 to the next `.then()`
})
.then((result) => {
    console.log("Final Result:", result); // Final Result: 17
});
```

Output:

A screenshot of a Node.js command prompt window. The title bar says "Node.js command prompt". The command prompt shows the command "C:\nodejs programs>node promise6.js" and the output: "Step 1: 2", "Step 2: 4", "Step 3: 12", and "Final Result: 17". The prompt then shows "C:\nodejs programs>".

```
C:\nodejs programs>node promise6.js
Step 1: 2
Step 2: 4
Step 3: 12
Final Result: 17
C:\nodejs programs>
```

Step-by-Step Explanation

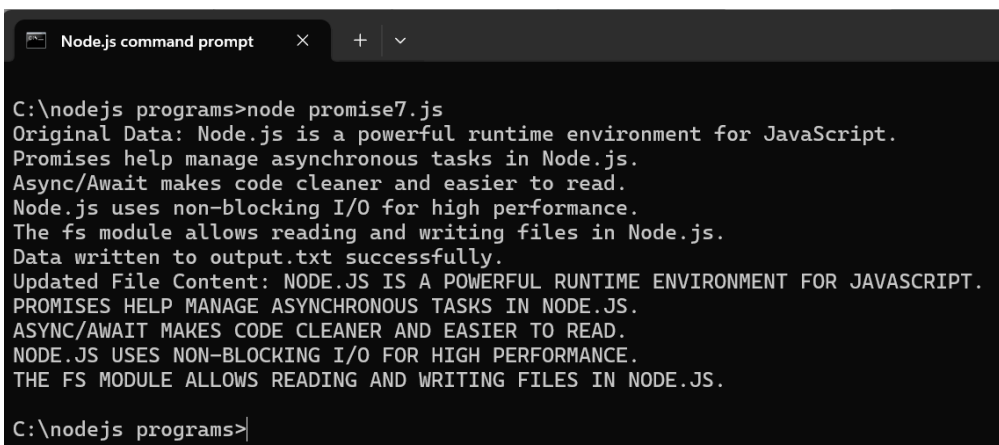
1. Initial Promise: The first `resolve(2)` begins the chain with a value of 2.
2. First `.then()`:
 - Receives 2 as the resolved value.
 - Logs it (Step 1: 2).
 - Returns $2 * 2$, which resolves to 4.
3. Second `.then()`:
 - Receives 4 from the previous `.then()`.
 - Logs it (Step 2: 4).
 - Returns $4 * 3$, which resolves to 12.
4. Third `.then()`:
 - Receives 12 from the previous `.then()`.
 - Logs it (Step 3: 12).
 - Returns $12 + 5$, which resolves to 17.
5. Final `.then()`:
 - Receives 17 and logs the final result.

Using Promise Chaining with Asynchronous Operations

Let's see how Promise chaining works in real-life scenarios, such as reading and modifying data from a file system.

```
const fs = require("fs").promises;
// Chaining multiple file operations
fs.readFile("input.txt", "utf8")
  .then((data) => {
    console.log("Original Data:", data);
    const updatedData = data.toUpperCase();
    return fs.writeFile("output.txt", updatedData); // Write updated data
  })
  .then(() => {
    console.log("Data written to output.txt successfully.");
    return fs.readFile("output.txt", "utf8"); // Read the updated file
  })
  .then((newData) => {
    console.log("Updated File Content:", newData);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise7.js
Original Data: Node.js is a powerful runtime environment for JavaScript.
Promises help manage asynchronous tasks in Node.js.
Async/Await makes code cleaner and easier to read.
Node.js uses non-blocking I/O for high performance.
The fs module allows reading and writing files in Node.js.
Data written to output.txt successfully.
Updated File Content: NODE.JS IS A POWERFUL RUNTIME ENVIRONMENT FOR JAVASCRIPT.
PROMISES HELP MANAGE ASYNCHRONOUS TASKS IN NODE.JS.
ASYNC/AWAIT MAKES CODE CLEANER AND EASIER TO READ.
NODE.JS USES NON-BLOCKING I/O FOR HIGH PERFORMANCE.
THE FS MODULE ALLOWS READING AND WRITING FILES IN NODE.JS.
C:\nodejs programs>
```

Handling Errors in a Promise Chain

Errors in a Promise chain are automatically propagated down the chain until a `.catch()` block is encountered. This centralizes error handling and avoids writing repetitive error-handling logic.

Example:

```
new Promise((resolve, reject) => {
  resolve(10);
})
  .then((result) => {
    console.log("Step 1:", result); // Step 1: 10
    return result / 2;
  })
  .then((result) => {
    console.log("Step 2:", result); // Step 2: 5
    throw new Error("Something went wrong!"); // Intentionally causing an error
  })
  .then((result) => {
```



```

    console.log("Step 3:", result); // This will not execute
  })
  .catch((error) => {
    console.error("Caught an Error:", error.message); // Handles the error
  });

```

Output:

```

Node.js command prompt
C:\nodejs programs>node promise8.js
Step 1: 10
Step 2: 5
Caught an Error: Something went wrong!

C:\nodejs programs>

```

Returning Another Promise in the Chain

Each `.then()` can return either a value or another Promise. When another Promise is returned, the chain waits for the new Promise to resolve before continuing.

Example:

```

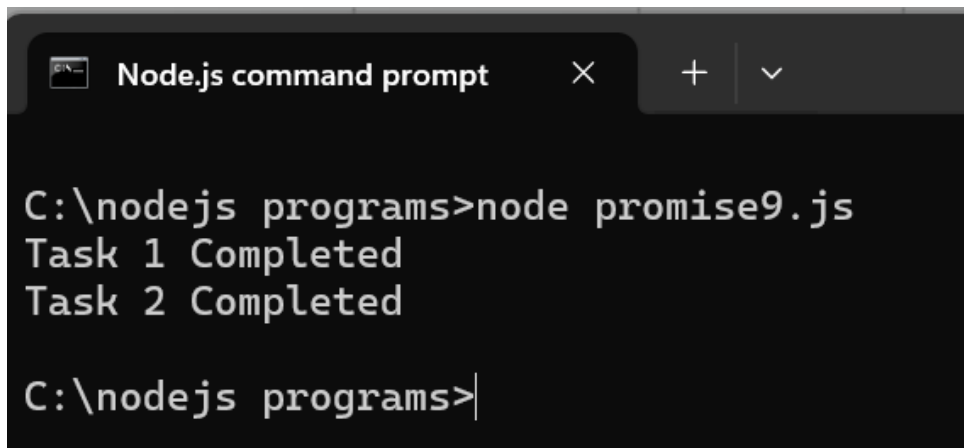
function asyncTask1() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Task 1 Completed");
    }, 1000); // Simulate a 1-second delay
  });
}

function asyncTask2() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Task 2 Completed");
    }, 2000); // Simulate a 2-second delay
  });
}

asyncTask1()
  .then((result) => {
    console.log(result);
    return asyncTask2(); // Return another Promise
  })
  .then((result) => {
    console.log(result); // Waits for asyncTask2 to complete
  });

```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise9.js
Task 1 Completed
Task 2 Completed

C:\nodejs programs>
```

Parallel Execution vs Chaining

While chaining is useful for sequential execution, sometimes you may want to run tasks in parallel. For parallel execution, you can use `Promise.all()` or `Promise.allSettled()` instead of chaining.

Advantages of Promise Chaining

1. **Readability:** Chaining avoids the deeply nested structure of callbacks, making the code cleaner.
2. **Error Propagation:** Errors bubble up to a single `.catch()` block, simplifying error handling.
3. **Sequential Execution:** Promises ensure operations execute in a controlled order.
4. **Modularity:** Each step in the chain is a reusable and testable unit.

Summary of Promise Chaining

- Each `.then()` passes its resolved value to the next `.then()`.
- Error handling is centralized with `.catch()`.
- Returning another Promise from `.then()` ensures that asynchronous operations are executed in sequence.

By mastering Promise chaining, you can handle complex workflows and asynchronous tasks in a clean, organized, and error-resistant way. This approach is foundational to writing modern, scalable Node.js applications!

IV. Parallel Promises

Parallel promises allow multiple asynchronous tasks to run **simultaneously** without waiting for one to complete before starting the next. This approach optimizes performance by making efficient use of system resources like I/O operations, networking, or processing tasks. In contrast to Promise chaining, where tasks are executed sequentially, parallel execution achieves faster results by running tasks concurrently.

Real-Life Scenarios Where Parallel Promises Are Useful

- Fetching data from multiple APIs simultaneously.
- Reading or writing multiple files in the file system.
- Executing database queries in parallel.
- Processing multiple images, videos, or other files concurrently.

Key Methods for Parallel Promises

Node.js provides specific methods to handle parallel promises effectively:

1. `Promise.all()`

- Waits for all Promises in an array to resolve.
- Resolves with an array of all resolved values.
- If any Promise in the array rejects, the entire `Promise.all()` rejects immediately.

2. Promise.allSettled()

- Waits for all Promises to settle (either resolved or rejected).
- Resolves with an array of objects describing each Promise's outcome (fulfilled or rejected).

3. Promise.race()

- Resolves or rejects as soon as the first Promise in the array settles (either resolved or rejected).

4. Promise.any()

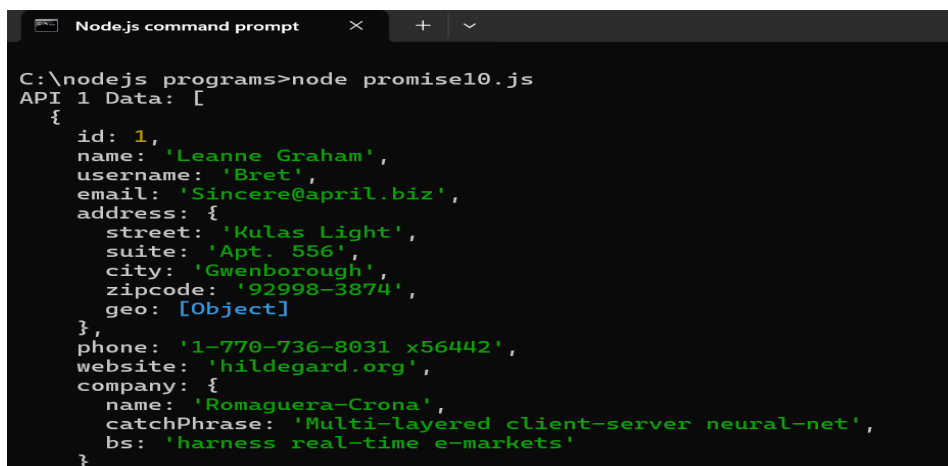
- Resolves as soon as the first Promise resolves.
- If all Promises reject, it rejects with an AggregateError.

Example of Parallel Execution with Promise.all()

Here's an example of running multiple tasks in parallel using Promise.all():

```
const fetch = require("node-fetch");    //for node v22 we don't need to import it
//API 1
function fetchAPI1() {
    return fetch("https://jsonplaceholder.typicode.com/users").then((res) => res.json());
}
//API 2
function fetchAPI2() {
    return fetch("https://jsonplaceholder.typicode.com/posts").then((res) => res.json());
}
//API 3
function fetchAPI3() {
    return fetch("https://jsonplaceholder.typicode.com/comments").then((res) =>
res.json());
}
// Run all APIs in parallel
Promise.all([fetchAPI1(), fetchAPI2(), fetchAPI3()])
    .then((results) => {
        console.log("API 1 Data:", results[0]);
        console.log("API 2 Data:", results[1]);
        console.log("API 3 Data:", results[2]);
    })
    .catch((error) => {
        console.error("Error in one of the APIs:", error);
    });
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise10.js
API 1 Data: [
  {
    id: 1,
    name: 'Leanne Graham',
    username: 'Bret',
    email: 'Sincere@april.biz',
    address: {
      street: 'Kulas Light',
      suite: 'Apt. 556',
      city: 'Gwenborough',
      zipcode: '92998-3874',
      geo: [Object]
    },
    phone: '1-770-736-8031 x56442',
    website: 'hildegard.org',
    company: {
      name: 'Romaguera-Crona',
      catchPhrase: 'Multi-layered client-server neural-net',
      bs: 'harness real-time e-markets'
    }
  }
]
```

Explanation:

- All three API calls (fetchAPI1, fetchAPI2, fetchAPI3) start simultaneously.
- Promise.all() waits for all of them to resolve.
- The resolved values are returned as an array.

Handling Errors in Promise.all()

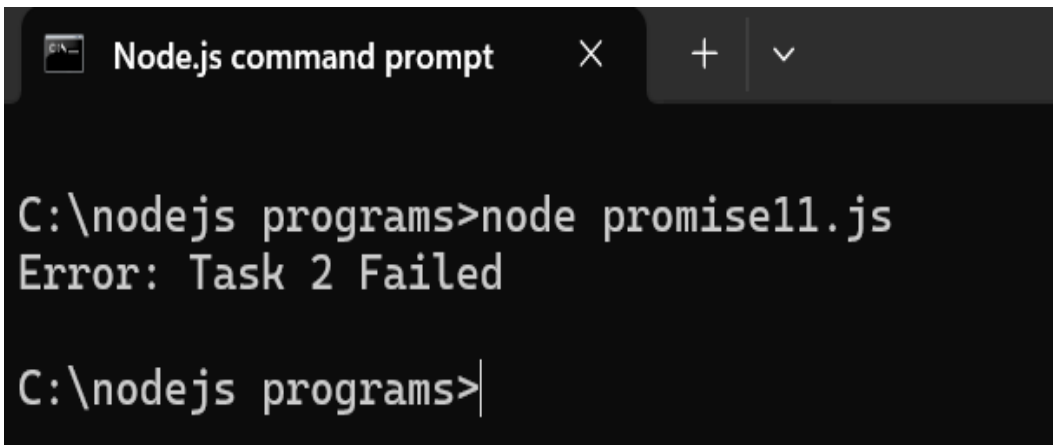
If one of the Promises rejects, the entire Promise.all() fails.

Example:

```
const task1 = Promise.resolve("Task 1 Completed");
const task2 = Promise.reject("Task 2 Failed");
const task3 = Promise.resolve("Task 3 Completed");

Promise.all([task1, task2, task3])
  .then((results) => {
    console.log(results); // Will not execute
  })
  .catch((error) => {
    console.error("Error:", error); // Logs: "Error: Task 2 Failed"
  });
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise11.js
Error: Task 2 Failed
C:\nodejs programs>
```

Using Promise.allSettled() for Parallel Promises

Unlike Promise.all(), Promise.allSettled() does not fail if one of the Promises rejects. Instead, it waits for all Promises to settle and provides their individual outcomes.

Example:

```
const task1 = Promise.resolve("Task 1 Completed");
const task2 = Promise.reject("Task 2 Failed");
const task3 = Promise.resolve("Task 3 Completed");

Promise.allSettled([task1, task2, task3])
  .then((results) => {
    console.log(results);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:

```
Node.js command prompt
C:\nodejs programs>node promise12.js
[
  { status: 'fulfilled', value: 'Task 1 Completed' },
  { status: 'rejected', reason: 'Task 2 Failed' },
  { status: 'fulfilled', value: 'Task 3 Completed' }
]
C:\nodejs programs>
```

Advantages of Promise.allSettled():

- You can handle both resolved and rejected Promises without stopping execution.
- Useful when you want to process as much as possible, even if some tasks fail.

Using Promise.race() for Parallel Promises

Promise.race() resolves or rejects as soon as the first Promise in the array settles (either resolved or rejected).

Example:

```
const slowTask = new Promise((resolve) => setTimeout(() => resolve("Slow Task"), 3000));
const fastTask = new Promise((resolve) => setTimeout(() => resolve("Fast Task"), 1000));
Promise.race([slowTask, fastTask])
  .then((result) => {
    console.log("Winner:", result); // Output: "Winner: Fast Task"
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:

```
Node.js command prompt
C:\nodejs programs>node promise12.js
Winner: Fast Task
C:\nodejs programs>
```

Key Use Case:

- Useful for tasks where you care only about the fastest result (e.g., querying multiple servers for the same data).

Using Promise.any() for Parallel Promises

Promise.any() resolves as soon as the first Promise resolves. If all Promises reject, it rejects with an AggregateError.

Example:

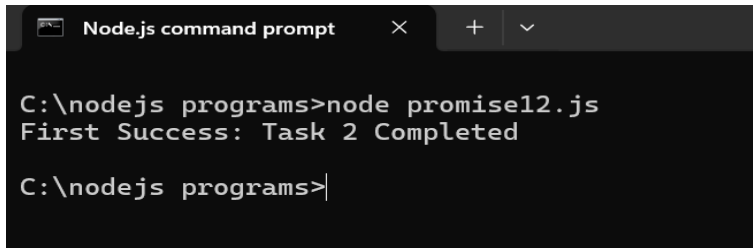
```
const task1 = Promise.reject("Task 1 Failed");
const task2 = Promise.resolve("Task 2 Completed");
const task3 = Promise.resolve("Task 3 Completed");
```

```

Promise.any([task1, task2, task3])
  .then((result) => {
    console.log("First Success:", result); // Output: "First Success: Task 2 Completed"
  })
  .catch((error) => {
    console.error("All Promises Failed:", error);
  });

```

Output:



```

Node.js command prompt
C:\nodejs programs>node promise12.js
First Success: Task 2 Completed
C:\nodejs programs>

```

Key Use Case:

- Ideal for scenarios where you need at least one successful result.

Practical Example: Reading Multiple Files in Parallel

Here's how you can read multiple files in parallel using Promise.all():

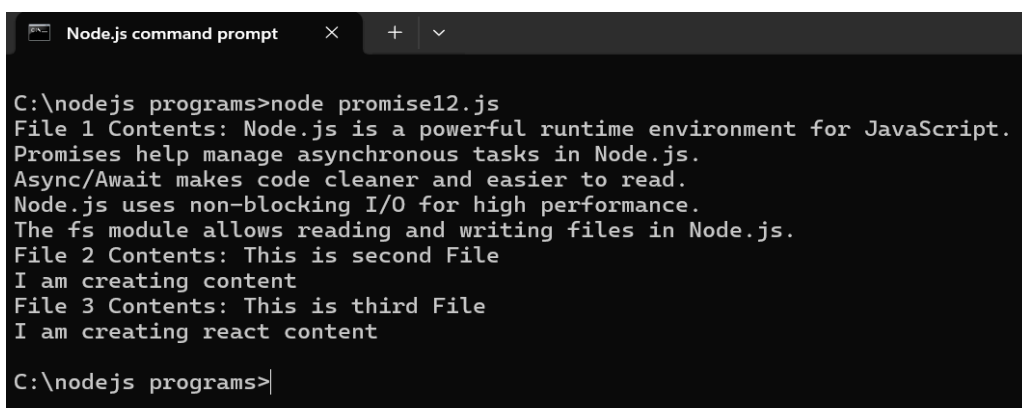
```

const fs = require("fs").promises;
const file1 = "file1.txt";
const file2 = "file2.txt";
const file3 = "file3.txt";

Promise.all([
  fs.readFile(file1, "utf8"),
  fs.readFile(file2, "utf8"),
  fs.readFile(file3, "utf8"),
])
  .then((contents) => {
    console.log("File 1 Contents:", contents[0]);
    console.log("File 2 Contents:", contents[1]);
    console.log("File 3 Contents:", contents[2]);
  })
  .catch((error) => {
    console.error("Error Reading Files:", error);
  });

```

Output:



```

Node.js command prompt
C:\nodejs programs>node promise12.js
File 1 Contents: Node.js is a powerful runtime environment for JavaScript.
Promises help manage asynchronous tasks in Node.js.
Async/Await makes code cleaner and easier to read.
Node.js uses non-blocking I/O for high performance.
The fs module allows reading and writing files in Node.js.
File 2 Contents: This is second File
I am creating content
File 3 Contents: This is third File
I am creating react content
C:\nodejs programs>

```

Performance Comparison: Sequential vs. Parallel

Sequential Execution:

```
fs.readFile("file1.txt", "utf8")
  .then((content1) => fs.readFile("file2.txt", "utf8"))
  .then((content2) => fs.readFile("file3.txt", "utf8"))
  .then((content3) => {
    console.log("All files read sequentially.");
  });
```

- Each file is read after the previous one completes.
- Total time = Sum of all file read times.

Parallel Execution:

```
Promise.all([
  fs.readFile("file1.txt", "utf8"),
  fs.readFile("file2.txt", "utf8"),
  fs.readFile("file3.txt", "utf8"),
])
  .then((contents) => {
    console.log("All files read in parallel.");
  });
```

- All files are read simultaneously.
- Total time = Longest individual file read time.

Advantages of Parallel Promises

1. Improved Performance: Simultaneous execution of tasks reduces total wait time.
2. Efficient Resource Utilization: Optimizes system resources like CPU, I/O, or network.
3. Simplified Code: Clean and concise handling of multiple asynchronous operations.

When to Use Parallel Promises

- Use `Promise.all()` when you need all tasks to succeed.
- Use `Promise.allSettled()` when you need results from all tasks, even if some fail.
- Use `Promise.race()` when you need the result of the fastest task.
- Use `Promise.any()` when you need at least one successful task.

By understanding and using parallel Promises effectively, you can optimize the performance and responsiveness of your Node.js applications.

V. `async/await` for Cleaner Asynchronous Code

`async/await` is a modern syntax introduced in ES2017 (ES8) that simplifies writing and reading asynchronous code. It allows you to write asynchronous code that looks synchronous, making it easier to understand and debug.

- `async`: Declares a function as asynchronous, automatically returning a Promise.
- `await`: Pauses the execution of an `async` function until the Promise is resolved or rejected.

Why Use `async/await`?

Before `async/await`, developers used callbacks or Promises for handling asynchronous tasks, leading to:

- **Callback Hell**: A situation where nested callbacks make the code hard to read and debug.
- **Complex Promise Chains**: Managing multiple `.then()` and `.catch()` blocks can be cumbersome.

`async/await` Advantages:

- Makes asynchronous code easier to read and maintain.
- Reduces boilerplate code compared to .then() chains.
- Handles errors in a cleaner way using try/catch.

Basics of async/await

- **Declaring an async Function**

An async function always returns a Promise. If the function returns a value, the Promise resolves with that value. If it throws an error, the Promise rejects with that error.

```
async function example() {
  return "Hello, Anuj!";
}
```

```
example().then((result) => console.log(result));
```

- **Using await**

The await keyword can only be used inside an async function. It waits for a Promise to resolve and then returns the resolved value.

```
async function fetchData() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
  console.log(data);
}
```

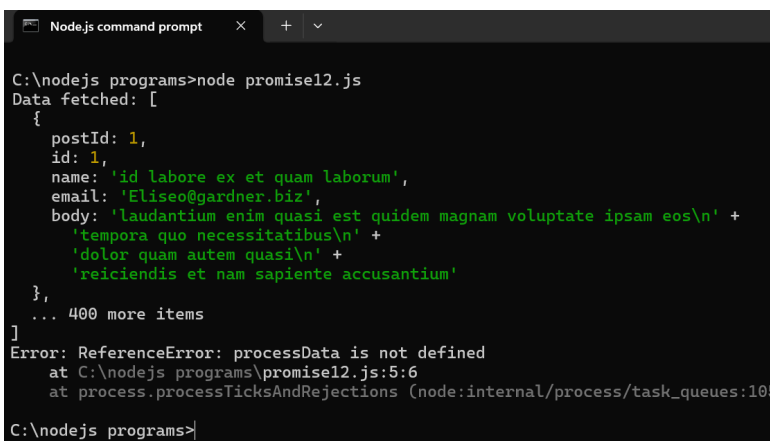
Example: Sequential Asynchronous Code with async/await

Let's take a real-world example where we fetch data from an API, process it, and then log it.

Without async/await (using .then()):

```
fetch("https://jsonplaceholder.typicode.com/comments")
  .then((response) => response.json())
  .then((data) => {
    console.log("Data fetched:", data);
    return processData(data);
  })
  .then((processedData) => {
    console.log("Processed Data:", processedData);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

Output:



```

C:\nodejs programs>node promise12.js
Data fetched: [
  {
    postId: 1,
    id: 1,
    name: 'id labore ex et quam laborum',
    email: 'Eliseo@gardner.biz',
    body: 'laudantium enim quasi est quidem magnam voluptate ipsam eos' +
      'tempora quo necessitatibus' +
      'dolor quam autem quasi' +
      'reiciendis et nam sapiente accusantium'
  },
  ... 400 more items
]
Error: ReferenceError: processData is not defined
    at C:\nodejs programs\promise12.js:5:6
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5)
C:\nodejs programs>

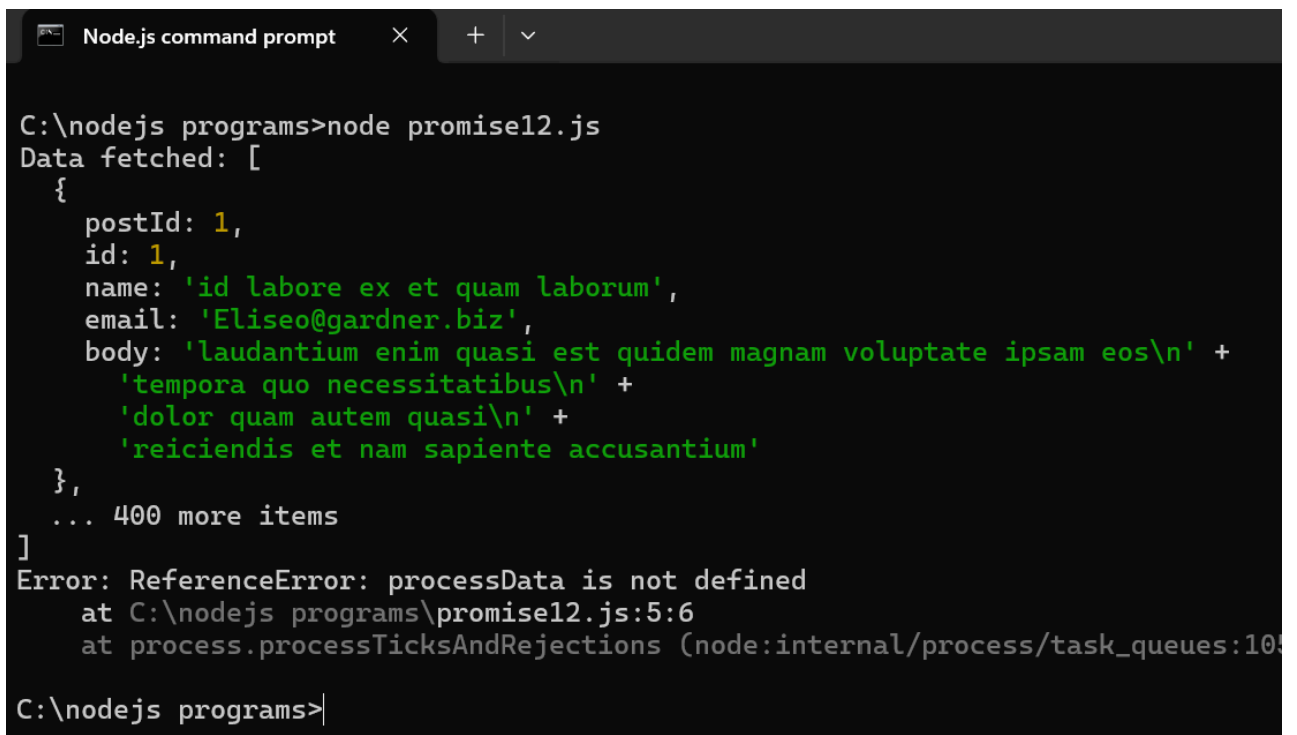
```


With async/await:

```
async function fetchAndProcessData() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/comments");
    const data = await response.json();
    console.log("Data fetched:", data);

    const processedData = await processData(data);
    console.log("Processed Data:", processedData);
  } catch (error) {
    console.error("Error:", error);
  }
}
fetchAndProcessData();
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise12.js
Data fetched: [
  {
    postId: 1,
    id: 1,
    name: 'id labore ex et quam laborum',
    email: 'Eliseo@gardner.biz',
    body: 'laudantium enim quasi est quidem magnam voluptate ipsam eos\
tempora quo necessitatibus\
dolor quam autem quasi\
reiciendis et nam sapiente accusantium'
  },
  ... 400 more items
]
Error: ReferenceError: processData is not defined
    at C:\nodejs programs\promise12.js:5:6
    at process.processTicksAndRejections (node:internal/process/task_queues:10:
C:\nodejs programs>
```

Key Differences:

- async/await eliminates the nested .then() structure, making the code easier to read.
- Error handling is centralized using try/catch.

Error Handling with async/await

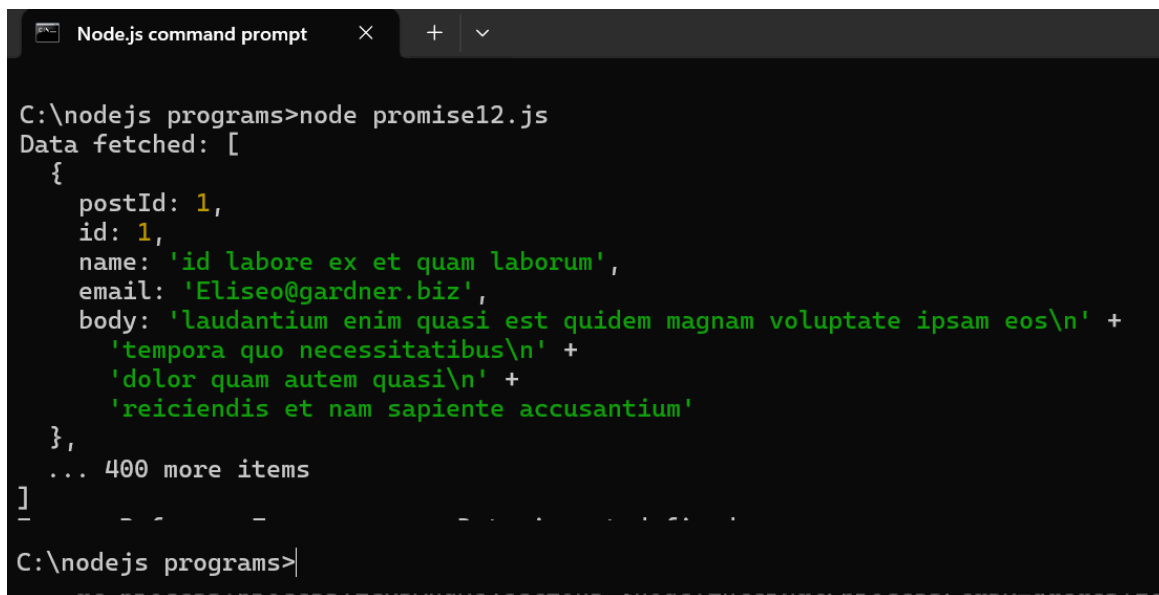
Errors in async functions can be caught using try/catch. This makes error handling much cleaner compared to chaining .catch() with Promises.

Example:

```
async function fetchData() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/comments");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Failed to fetch data:", error.message);
  }
}
```

```
}  
fetchData();
```

Output:



```
Node.js command prompt
C:\nodejs programs>node promise12.js
Data fetched: [
  {
    postId: 1,
    id: 1,
    name: 'id labore ex et quam laborum',
    email: 'Eliseo@gardner.biz',
    body: 'laudantium enim quasi est quidem magnam voluptate ipsam eos\n' +
      'tempora quo necessitatibus\n' +
      'dolor quam autem quasi\n' +
      'reiciendis et nam sapiente accusantium'
  },
  ... 400 more items
]
```

Parallel Execution with async/await

While async/await works well for sequential tasks, running tasks in parallel requires additional care. You can use `Promise.all()` in combination with async/await.

Example:

```
async function fetchMultipleAPIs() {  
  try {  
    const [data1, data2, data3] = await Promise.all([  
      fetch("https://api.example.com/data1").then((res) => res.json()),  
      fetch("https://api.example.com/data2").then((res) => res.json()),  
      fetch("https://api.example.com/data3").then((res) => res.json())  
    ]);  
  
    console.log("API 1 Data:", data1);  
    console.log("API 2 Data:", data2);  
    console.log("API 3 Data:", data3);  
  } catch (error) {  
    console.error("Error in one of the API calls:", error);  
  }  
}
```

```
fetchMultipleAPIs();
```

Output:

How It Works:

- All the API calls start simultaneously because of `Promise.all()`.
- The `await` keyword waits for all the Promises to resolve.
- The results are destructured into individual variables.

Avoiding Sequential Overhead

Using `await` in a loop may lead to sequential execution, which can be inefficient.

Sequential Execution Example:

```
async function processTasks(tasks) {
```

```

    for (const task of tasks) {
      const result = await task(); // Tasks run one after another
      console.log(result);
    }
  }
}

```

Parallel Execution Example:

```

async function processTasksInParallel(tasks) {
  const results = await Promise.all(tasks.map((task) => task())); // All tasks run concurrently
  console.log(results);
}

```

Key Difference:

- Sequential execution waits for each task to complete before starting the next.
- Parallel execution runs all tasks concurrently, improving performance.

Combining async/await with Other Promise Methods

async/await is compatible with other Promise methods like Promise.race() or Promise.any().

Example with Promise.race():

```

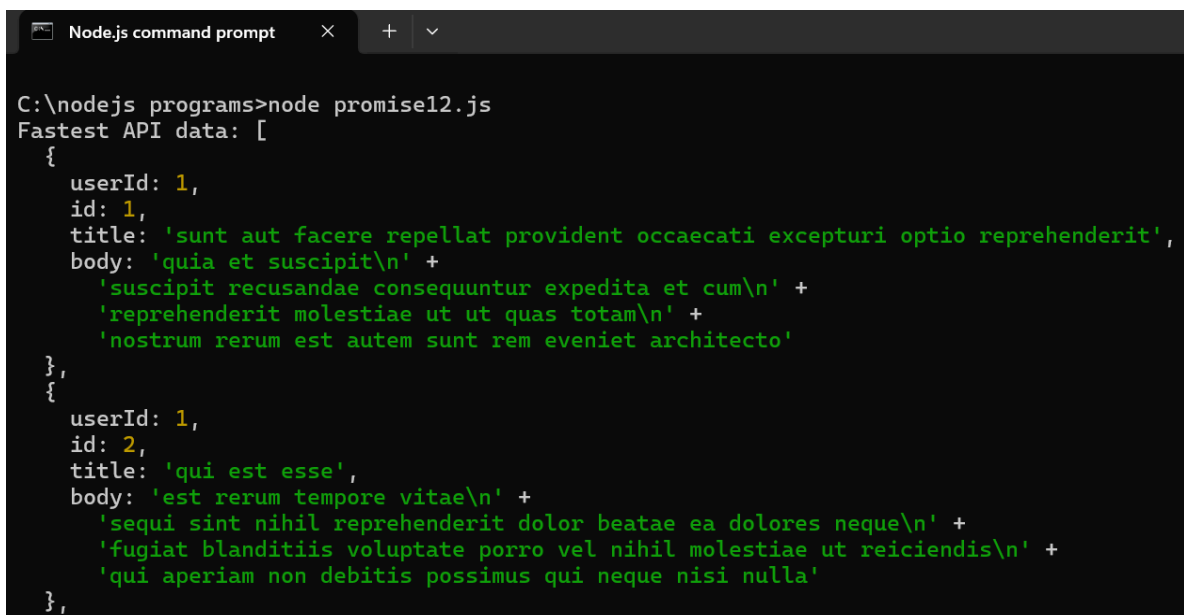
async function fetchFastestAPI() {
  try {
    const fastestResponse = await Promise.race([
      fetch("https://jsonplaceholder.typicode.com/comments"),
      fetch("https://jsonplaceholder.typicode.com/posts")
    ]);

    const data = await fastestResponse.json();
    console.log("Fastest API data:", data);
  } catch (error) {
    console.error("Error:", error);
  }
}

fetchFastestAPI();

```

Output:



```

Node.js command prompt
C:\nodejs programs>node promise12.js
Fastest API data: [
  {
    userId: 1,
    id: 1,
    title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit',
    body: 'quia et suscipit\n' +
      'suscipit recusandae consequuntur expedita et cum\n' +
      'reprehenderit molestiae ut ut quas totam\n' +
      'nostrum rerum est autem sunt rem eveniet architecto'
  },
  {
    userId: 1,
    id: 2,
    title: 'qui est esse',
    body: 'est rerum tempore vitae\n' +
      'sequi sint nihil reprehenderit dolor beatae ea dolores neque\n' +
      'fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\n' +
      'qui aperiam non debitis possimus qui neque nisi nulla'
  }
]

```

Common Pitfalls and Best Practices

1. Do Not Use await Outside an async Function:
// Incorrect:
`const result = await fetch("https://api.example.com/data");` // SyntaxError
2. Avoid Blocking Execution with await in Loops: Prefer Promise.all() for parallel execution.
3. Graceful Error Handling: Use try/catch for synchronous-looking error handling.
4. Use await Sparingly: Avoid overusing await, especially in performance-critical sections. Combine it with Promise methods where necessary.

Real-Life Application Example: File Processing

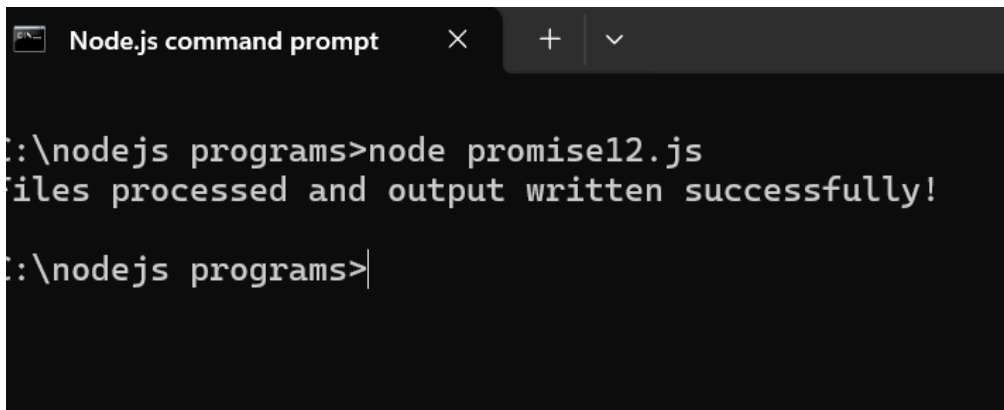
Here's a practical example of reading and writing multiple files using async/await:

```
const fs = require("fs").promises;
async function processFiles() {
  try {
    const file1Content = await fs.readFile("file1.txt", "utf8");
    const file2Content = await fs.readFile("file2.txt", "utf8");

    const combinedContent = `${file1Content}\n${file2Content}`;
    await fs.writeFile("output.txt", combinedContent);

    console.log("Files processed and output written successfully!");
  } catch (error) {
    console.error("Error processing files:", error.message);
  }
}
processFiles();
```

Output:

A screenshot of a Node.js command prompt window. The title bar reads "Node.js command prompt". The terminal shows the command `C:\nodejs programs>node promise12.js` being executed. The output is `Files processed and output written successfully!`. The prompt then returns to `C:\nodejs programs>`.

Why async/await is a Game-Changer

- **Code Readability:** Looks like synchronous code, reducing cognitive load.
- **Error Handling:** Simplifies debugging with try/catch.
- **Integration:** Works seamlessly with existing Promises.
- **Versatility:** Supports sequential and parallel tasks effortlessly.

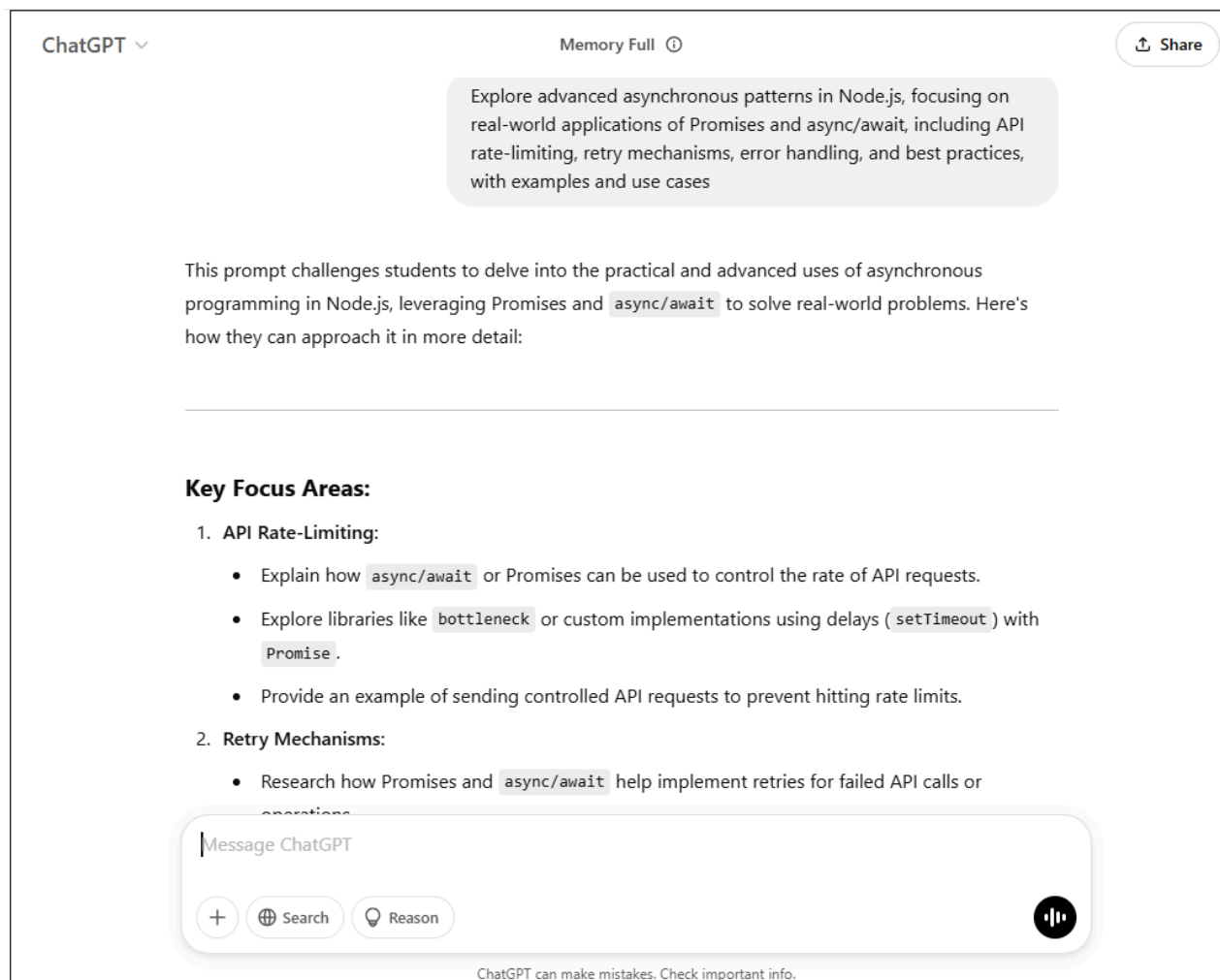
By using async/await, you can write robust, readable, and maintainable asynchronous code, taking full advantage of Node.js's non-blocking architecture.

Exercise:

Use GPT and explore advanced asynchronous patterns in Node.js.

Put the below problem statement in the message box and see what ChatGPT says.

“Explore advanced asynchronous patterns in Node.js, focusing on real-world applications of Promises and async/await, including API rate-limiting, retry mechanisms, error handling, and best practices, with examples and use cases.”



The screenshot shows a ChatGPT interface. At the top, there's a header with "ChatGPT" and a dropdown arrow, "Memory Full" with an info icon, and a "Share" button. Below the header, a prompt box contains the text: "Explore advanced asynchronous patterns in Node.js, focusing on real-world applications of Promises and async/await, including API rate-limiting, retry mechanisms, error handling, and best practices, with examples and use cases". The response area shows the start of an answer: "This prompt challenges students to delve into the practical and advanced uses of asynchronous programming in Node.js, leveraging Promises and `async/await` to solve real-world problems. Here's how they can approach it in more detail:". Below this, there's a section titled "Key Focus Areas:" followed by two numbered points. Point 1 is "API Rate-Limiting:" with three bullet points. Point 2 is "Retry Mechanisms:" with one bullet point. At the bottom, there's a message input box with the placeholder text "Message ChatGPT", a plus icon, a "Search" button, a "Reason" button, and a microphone icon. A footer note at the very bottom says "ChatGPT can make mistakes. Check important info."

ChatGPT

Memory Full ⓘ

Share

Explore advanced asynchronous patterns in Node.js, focusing on real-world applications of Promises and `async/await`, including API rate-limiting, retry mechanisms, error handling, and best practices, with examples and use cases

This prompt challenges students to delve into the practical and advanced uses of asynchronous programming in Node.js, leveraging Promises and `async/await` to solve real-world problems. Here's how they can approach it in more detail:

Key Focus Areas:

1. **API Rate-Limiting:**
 - Explain how `async/await` or Promises can be used to control the rate of API requests.
 - Explore libraries like `bottleneck` or custom implementations using delays (`setTimeout`) with `Promise`.
 - Provide an example of sending controlled API requests to prevent hitting rate limits.
2. **Retry Mechanisms:**
 - Research how Promises and `async/await` help implement retries for failed API calls or operations.

Message ChatGPT

+ Search Reason

ChatGPT can make mistakes. Check important info.

After the 1.5 hr ILT, the student has to do a 30 minute live lab. Please refer to the Session 5 Lab doc in the LMS.